

Refatoração Automatizada de Classes de Teste de Software Usando Métricas de Similaridade e Métodos de Agrupamento

Exame de Qualificação de Doutorado

Aluno

Lucas Pereira da Silva

Orientadora

Prof.^a Dr.^a Patrícia Vilain

Membros da banca

Prof. Dr. Adenilso da Silva Simão

Prof.^a Dr.^a Carina Friedrich Dorneles

Prof.^a Dr.^a Juliana Silva Herbert

Roteiro

- Introdução
- Problema de Pesquisa
- Hipótese de Pesquisa
- Objetivos do Trabalho
- Fundamentação Teórica
- Estado da Arte
- Proposta
- Avaliação
- Conclusões

Roteiro

- Introdução
- Problema de Pesquisa
- Hipótese de Pesquisa
- Objetivos do Trabalho
- Fundamentação Teórica
- Estado da Arte
- Proposta
- Avaliação
- Conclusões

Introdução

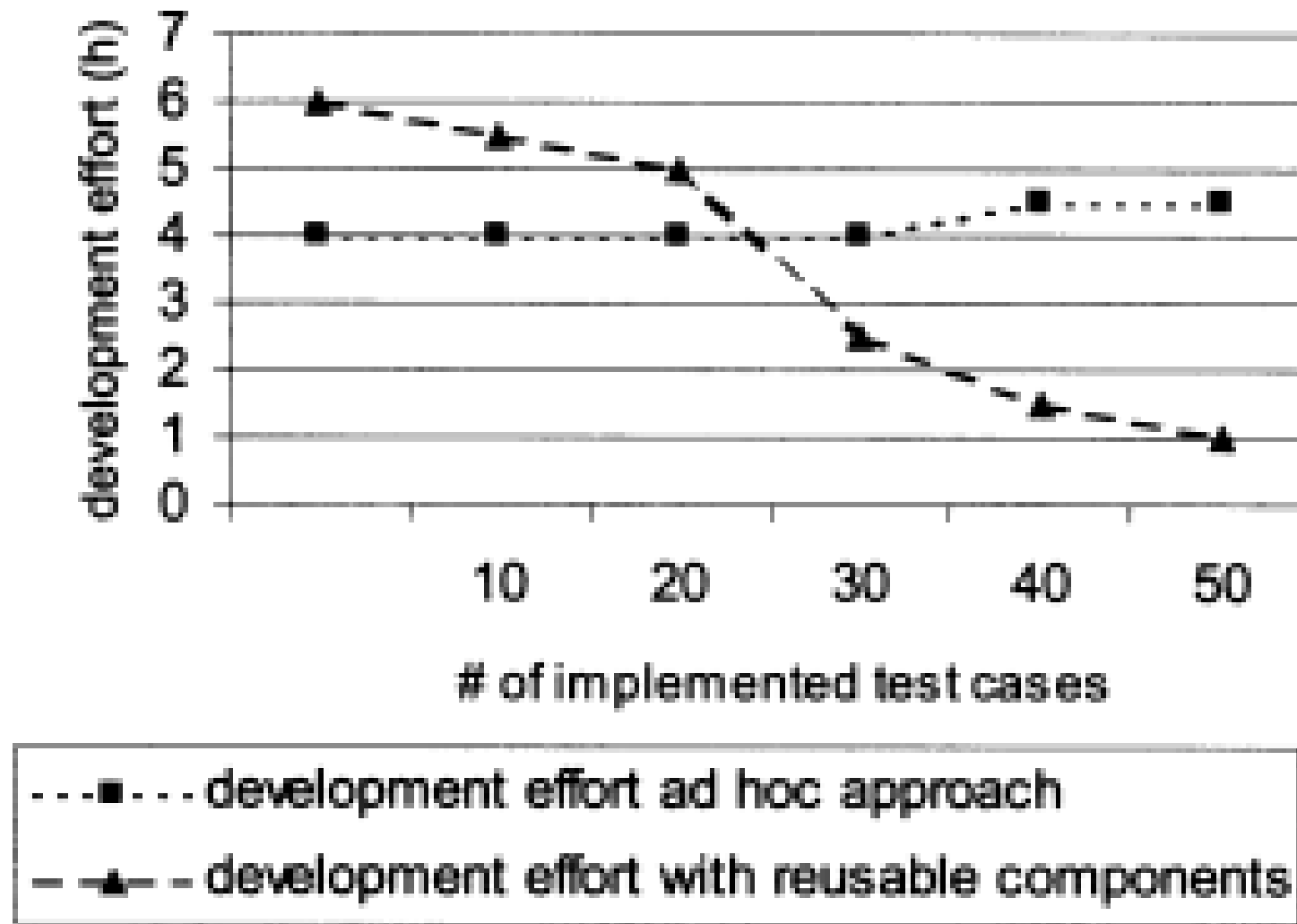
- Teste de *software*:
 - Atividade do processo de desenvolvimento e manutenção de *software*.
- Existem diferentes formas para automatizar testes:
 - **Codificação manual de scripts de teste.**
 - Geração de scripts a partir de gravações do SUT (*System Under Test*).
 - Geração baseada em modelo (*Model Based Testing* – MBT).
- Este trabalho se encaixa no contexto de testes codificados manualmente.
- O código de teste precisa ser mantido durante todo o projeto.
- Pode consumir de 60% a 80% do esforço (Tasai *et al.*, 2003).
- O sucesso a longo prazo é influenciado pela manutenibilidade.

```
1 public class TesteSaqueValorDoSaldo() {
2     @Test
3     public void sacarValorDoSaldo() {
4         CasaDaMoeda casaDaMoeda = new CasaDaMoeda(Moeda.BRL);
5         SistemaBancario sistema = new SistemaBancario();
6         Banco bb = sistema.criarBanco("Banco do Brasil", Moeda.BRL);
7         Agencia trindade = bb.criarAgencia("Trindade");
8         Conta joao = trindade.criarConta("João", "000.000.000-00");
9         Dinheiro cincoReais = casaDaMoeda.emitir(5);
10        joao.depositar(cincoReais);
11        Transacao saque = joao.sacar(cincoReais);
12        assertTrue(saque.sucesso());
13        assertEquals(casaDaMoeda.zero(), joao.calcularSaldo());
14    }
15 }
```

```
1 public class TesteSaqueMaiorQue0Saldo() {
2     @Test
3     public void sacarMaisQue0Saldo() {
4         CasaDaMoeda casaDaMoeda = new CasaDaMoeda(Moeda.BRL);
5         SistemaBancario sistema = new SistemaBancario();
6         Banco bb = sistema.criarBanco("Banco do Brasil", Moeda.BRL);
7         Agencia trindade = bb.criarAgencia("Trindade");
8         Conta joao = trindade.criarConta("João", "000.000.000-00");
9         Dinheiro cincoReais = casaDaMoeda.emitir(5);
10        joao.depositar(cincoReais);
11        Dinheiro dezReais = casaDaMoeda.emitir(10);
12        Transacao saque = joao.sacar(dezReais);
13        assertFalse(saque.sucesso());
14        assertEquals(cincoReais, joao.calcularSaldo());
15    }
16 }
```

Introdução

- Necessário garantir uma boa manutenibilidade do código de teste.
- Técnicas de reuso de código reduzem a quantidade de duplicação.
- A tendência é reduzir o esforço de desenvolvimento a longo prazo.
- Porém possuem um esforço de implementação.



Introdução

- Reuso de código através da estratégia *implicit setup*.


```
1 public class TesteSaque {
2     private Conta joao;
3     private Dinheiro cincoReais;
4     private CasaDaMoeda casaDaMoeda;
5
6     @BeforeEach
7     public void configurar() {
8         CasaDaMoeda casaDaMoeda = new CasaDaMoeda(Moeda.BRL);
9         SistemaBancario sistema = new SistemaBancario();
10        Banco bb = sistema.criarBanco("Banco do Brasil", Moeda.BRL);
11        Agencia trindade = bb.criarAgencia("Trindade");
12        joao = trindade.criarConta("João", "000.000.000-00");
13        cincoReais = casaDaMoeda.emitir(5);
14        joao.depositar(cincoReais);
15    }
16
17    @Test
18    public void sacarValorDoSaldo() {
19        Transacao saque = joao.sacar(cincoReais);
20        assertTrue(saque.sucesso());
21        assertEquals(casaDaMoeda.zero(), joao.calcularSaldo());
22    }
23
24    @Test
25    public void sacarMaisQueOSaldo () {
26        Dinheiro dezReais = casaDaMoeda.emitir(10);
27        Transacao saque = joao.sacar(cincoReais);
28        assertFalse(saque.sucesso());
29        assertEquals(cincoReais, joao.calcularSaldo());
30    }
31 }
```

Introdução

- De 8% a 42% dos testes são duplicações (Landhäußer e Tichy, 2012).
- Desenvolvedor precisa manualmente:
 - Identificar testes com código em comum.
 - Agrupar os testes com código em comum na mesma classe.
 - Refatorar manualmente o código em comum.
- Esforço para promover reuso aumenta com o número de testes.
- Refatoração constante pode ser muito dispendiosa.
- Nem sempre é simples identificar se a refatoração compensa.

```
1 public class TesteSaque {
2     private Conta joao;
3     private Dinheiro cincoReais;
4     private CasaDaMoeda casaDaMoeda;
5
6     @BeforeEach
7     public void configurar() {
8         CasaDaMoeda casaDaMoeda = new CasaDaMoeda(Moeda.BRL);
9         SistemaBancario sistema = new SistemaBancario();
10        Banco bb = sistema.criarBanco("Banco do Brasil", Moeda.BRL);
11        Agencia trindade = bb.criarAgencia("Trindade");
12        joao = trindade.criarConta("João", "000.000.000-00");
13        cincoReais = casaDaMoeda.emitir(5);
14        joao.depositar(cincoReais);
15    }
16
17    @Test
18    public void sacarValorDoSaldo() {
19        Transacao saque = joao.sacar(cincoReais);
20        assertTrue(saque.sucesso());
21        assertEquals(casaDaMoeda.zero(), joao.calcularSaldo());
22    }
23
24    @Test
25    public void sacarMaisQueOSaldo () {
26        Dinheiro dezReais = casaDaMoeda.emitir(10);
27        Transacao saque = joao.sacar(cincoReais);
28        assertFalse(saque.sucesso());
29        assertEquals(cincoReais, joao.calcularSaldo());
30    }
31 }
```

```
1 public class TesteSaqueDeChequeEspecial {
2     @Test
3     public void sacarDeChequeEspecial() {
4         CasaDaMoeda casaDaMoeda = new CasaDaMoeda(Moeda.BRL);
5         SistemaBancario sistema = new SistemaBancario();
6         Banco bb = sistema.criarBanco("Banco do Brasil", Moeda.BRL);
7         Agencia trindade = bb.criarAgencia("Trindade");
8         Conta joao = trindade.criarConta("João", "000.000.000-00");
9         Dinheiro cincoReais = casaDaMoeda.emitir(5);
10        joao.fixarLimiteDeChequeEspecial(cincoReais);
11        Transacao saque = joao.sacar(cincoReais);
12        assertTrue(saque.sucesso());
13        assertEquals(casaDaMoeda.zero(), joao.calcularSaldo());
14        assertEquals(cincoReais, joao.calcularGastoComChequeEspecial());
15    }
16 }
```

```
1 public class TesteSaque {
2     private Conta joao;
3     private Dinheiro cincoReais;
4     private CasaDaMoeda casaDaMoeda;
5
6     @BeforeEach
7     public void configurar() {
8         CasaDaMoeda casaDaMoeda = new CasaDaMoeda(Moeda.BRL);
9         SistemaBancario sistema = new SistemaBancario();
10        Banco bb = sistema.criarBanco("Banco do Brasil", Moeda.BRL);
11        Agencia trindade = bb.criarAgencia("Trindade");
12        joao = trindade.criarConta("João", "000.000.000-00");
13        cincoReais = casaDaMoeda.emitir(5);
14        joao.depositar(cincoReais);
15    }
16
17    @Test
18    public void sacarValorDoSaldo() {
19        Transacao saque = joao.sacar(cincoReais);
20        assertTrue(saque.sucesso());
21        assertEquals(casaDaMoeda.zero(), joao.calcularSaldo());
22    }
23
24    @Test
25    public void sacarMaisQueOSaldo () {
26        Dinheiro dezReais = casaDaMoeda.emitir(10);
27        Transacao saque = joao.sacar(cincoReais);
28        assertFalse(saque.sucesso());
29        assertEquals(cincoReais, joao.calcularSaldo());
30    }
31 }
```

Roteiro

- Introdução
- Problema de Pesquisa
- Hipótese de Pesquisa
- Objetivos do Trabalho
- Fundamentação Teórica
- Estado da Arte
- Proposta
- Avaliação
- Conclusões

Problema de Pesquisa

- Técnicas de reuso são usadas para melhorar a manutenibilidade.
- Nem sempre o ganho obtido na melhoria compensa o esforço.
- Erros manuais podem ocorrer durante a refatoração.
- Como reduzir a duplicação sem aumentar o esforço?
- E sem estar suscetível a erros de refatoração?

Roteiro

- Introdução
- Problema de Pesquisa
- Hipótese de Pesquisa
- Objetivos do Trabalho
- Fundamentação Teórica
- Estado da Arte
- Proposta
- Avaliação
- Conclusões

Hipótese de Pesquisa

- Métricas de similaridade para identificar código comum entre testes.
- Métodos de agrupamento para criar grupos (classes) de teste.
- Refatoração automatizada.
- Reduzir o esforço de refatoração do código de teste.
- Sem que o processo esteja sucessível a erros de refatoração.

Roteiro

- Introdução
- Problema de Pesquisa
- Hipótese de Pesquisa
- **Objetivos do Trabalho**
- Fundamentação Teórica
- Estado da Arte
- Proposta
- Avaliação
- Conclusões

Objetivos do Trabalho

- Reduzir o esforço total de desenvolvimento dos testes sem alterar o resultado esperado dos testes.

Objetivos do Trabalho

- Identificar métricas de similaridade adequadas para identificar testes com código em comum.
- Identificar métodos de agrupamento adequados para criar grupos de testes com código em comum entre si.
- Definir um método genérico para refatoração automatizada de classes de teste a partir da utilização das métricas de similaridade e métodos de agrupamento.
- Definir implementações específicas para o método genérico com base na seleção das métricas de similaridade e métodos de agrupamento mais adequados.

Roteiro

- Introdução
- Problema de Pesquisa
- Hipótese de Pesquisa
- Objetivos do Trabalho
- **Fundamentação Teórica**
- Estado da Arte
- Proposta
- Avaliação
- Conclusões

Fundamentação Teórica

- Métricas de similaridade possuem aplicações na computação.
- Grau de similaridade determina a similaridade entre dois elementos.
- Este trabalho é focado nas métricas de similaridade de código.
- Cinco grandes abordagens de similaridade de código.

Fundamentação Teórica

- Métricas baseadas em medidas sobre o código.
- Métricas baseadas em texto.
- Métricas baseadas em *tokens*.
- Métricas baseadas em árvores.
- Métricas baseadas em grafos.

Fundamentação Teórica

- Métodos de agrupamento:
 - Formação de grupos a partir de um conjunto de elementos.
- Maximizar a similaridade entre os elementos de um grupo.
- Minimizar a similaridade entre os elementos de grupos diferentes.
- Diferentes categorias de métodos de agrupamento.

Fundamentação Teórica

- Métodos por particionamento.
- Métodos hierárquicos.
- Métodos baseados em densidade.
- Métodos baseados em modelo.
- Métodos baseados em *grid*.

Roteiro

- Introdução
- Problema de Pesquisa
- Hipótese de Pesquisa
- Objetivos do Trabalho
- Fundamentação Teórica
- Estado da Arte
- Proposta
- Avaliação
- Conclusões

Estado da Arte

- Uso de métricas de similaridade no contexto de teste de *software*:
 - Redução de suíte de teste.
 - Priorização de casos de teste.
- Ledru *et al.* (2012).
- Miranda *et al.* (2018).
- Hemmati *et al.* (2010).

Estado da Arte

- Técnicas de reuso de código de teste.
- *Implicit setup* (Meszaros, 2007).
- *Delegate setup* (Meszaros, 2007).
- Picon (Longo *et al.*, 2015).
- DbUnit (Christensen *et al.*, 2006).
- Estória (da Silva e Vilain, 2017).
- Comparação das técnicas de reuso por diferentes fatores.

Estado da Arte

Técnica	Escopo de Refatoração
DbUnit (Christensen et al., 2006)	Entidades do banco de dados
Picon (Longoet al., 2015)	POJOs
Implicit setup (Meszaros, 2007)	Sequências iniciais do código de teste
Delegated setup (Meszaros, 2007)	Trechos contíguos do código de teste
Estória (da Silva e Vilain, 2017)	Sequências iniciais do código de teste
Proposta deste trabalho	Sequências iniciais do código de teste

Estado da Arte

Técnica	Forma de Identificação de Duplicação
DbUnit (Christensen et al., 2006)	Identificação manual de testes que envolvem as mesmas entidades do banco de dados
Picon (Longoet al., 2015)	Identificação manual de POJOs que são iguais entre si
Implicit setup (Meszaros, 2007)	Identificação manual de sequências iniciais do código de teste que são iguais entre si
Delegated setup (Meszaros, 2007)	Identificação manual de trechos contíguos do código de teste que são iguais entre si
Estória (da Silva e Vilain, 2017)	Identificação manual de sequências iniciais do código de teste que são iguais entre si
Proposta deste trabalho	Identificação automatizada de sequências iniciais do código de teste que são iguais entre si

Estado da Arte

Técnica	Processo de Refatoração
DbUnit (Christensen et al., 2006)	Definição manual dos testes envolvendo uma mesma entidade do banco de dados em uma mesma classe de teste
Picon (Longo et al., 2015)	Definição manual do objeto em um arquivo externo e declaração do objeto como atributo das classes de teste que o utilizam
Implicit setup (Meszaros, 2007)	Definição manual da sequência inicial do código de teste em uma classe e colocação dos testes que possuem a mesma sequência inicial do código de teste na mesma classe
Delegated setup (Meszaros, 2007)	Definição manual de um método contendo o trecho contíguo do código de teste e declaração de chamada de método nos testes que possuem o mesmo trecho contíguo de código
Estória (da Silva e Vilain, 2017)	Definição manual da sequência inicial do código de teste em uma classe e colocação dos testes que possuem a mesma sequência inicial do código de teste na mesma classe
Proposta deste trabalho	Definição automatizada da sequência inicial do código de teste em uma classe e colocação dos testes que possuem a mesma sequência inicial do código de teste na mesma classe

Estado da Arte

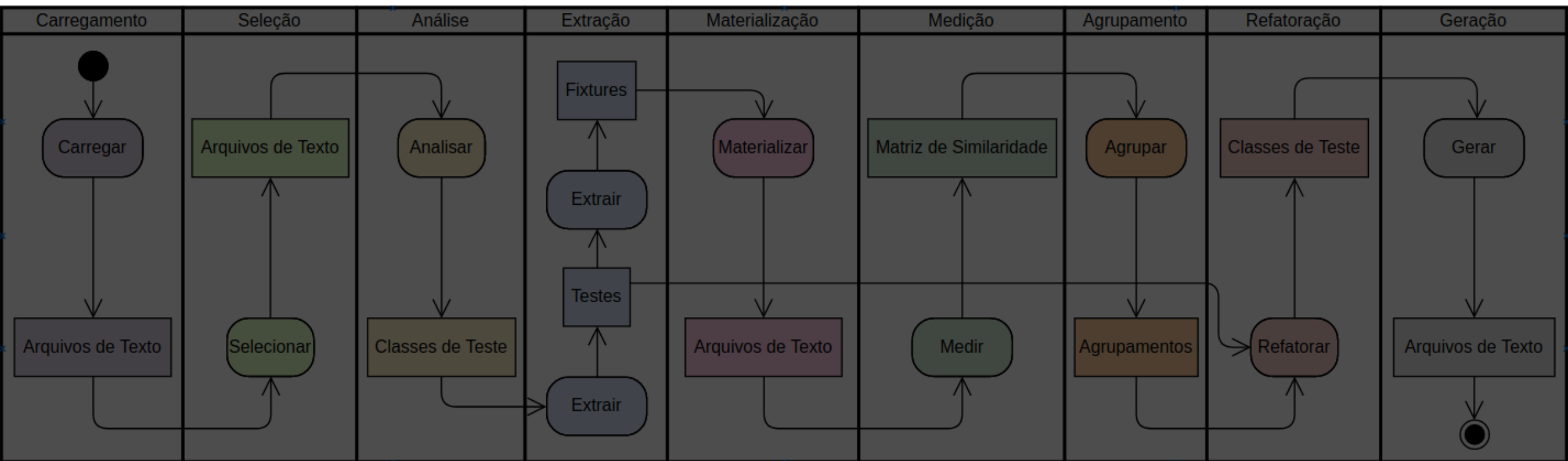
Técnica	Esforço de Refatoração	
	Quando Aumenta	Quando Diminuí
DbUnit (Christensen et al., 2006)	O número de entidades do banco de dados aumenta	O número de testes para uma mesma entidade do banco de dados aumenta
Picon (Longoet al., 2015)	A quantidade de objetos diferentes aumenta	A quantidade de objetos iguais aumenta
Implicit setup (Meszaros, 2007)	A quantidade de sequências iniciais do código de teste diferentes aumenta	A quantidade de sequências iniciais iguais do código aumenta.
Delegated setup (Meszaros, 2007)	A quantidade de trechos contíguos de código diferentes aumenta	A quantidade de trechos contíguos de código iguais aumenta
Estória (da Silva e Vilain, 2017)	A quantidade de sequências iniciais do código de teste diferentes aumenta	A quantidade de sequências iniciais iguais do código aumenta.
Proposta deste trabalho	Por ser automatizada não existe esforço de desenvolvimento	

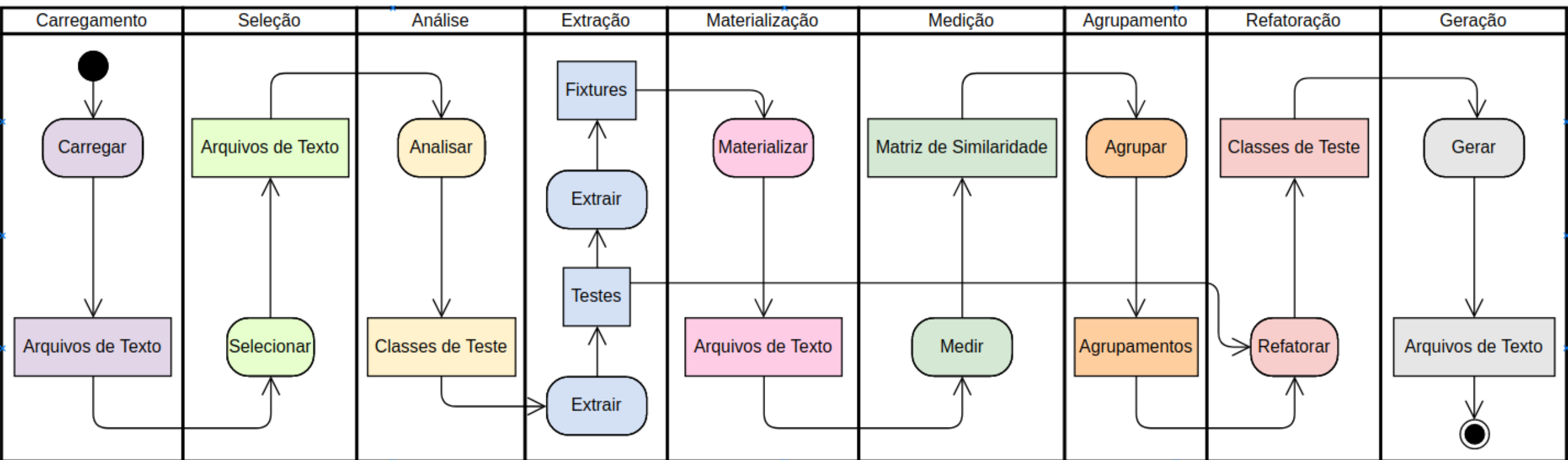
Roteiro

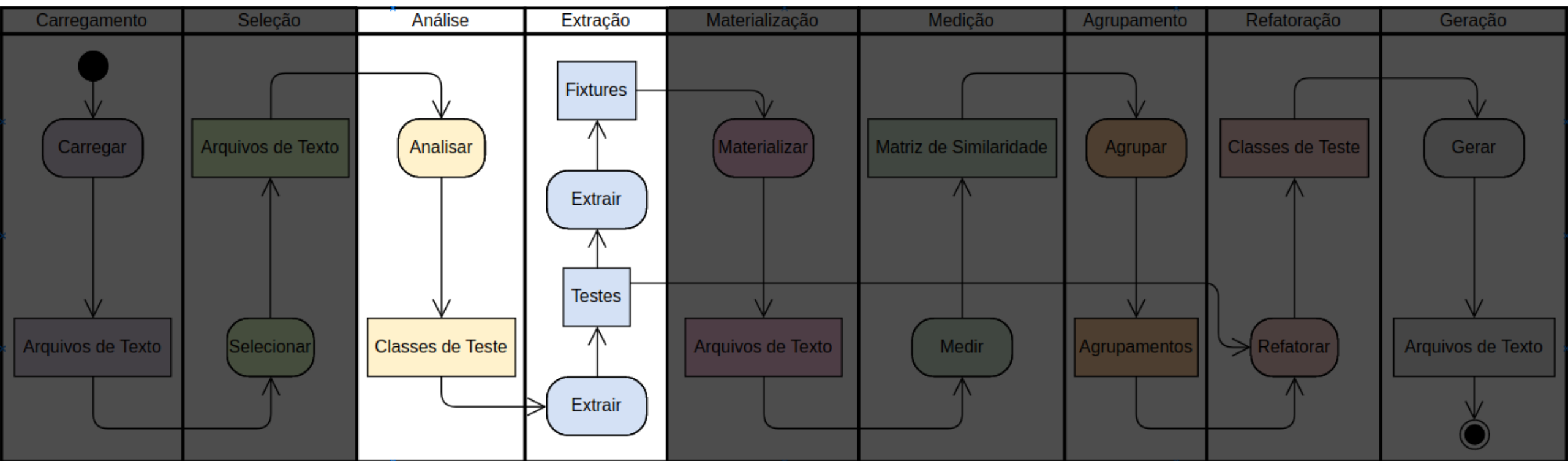
- Introdução
- Problema de Pesquisa
- Hipótese de Pesquisa
- Objetivos do Trabalho
- Fundamentação Teórica
- Estado da Arte
- **Proposta**
- Avaliação
- Conclusões

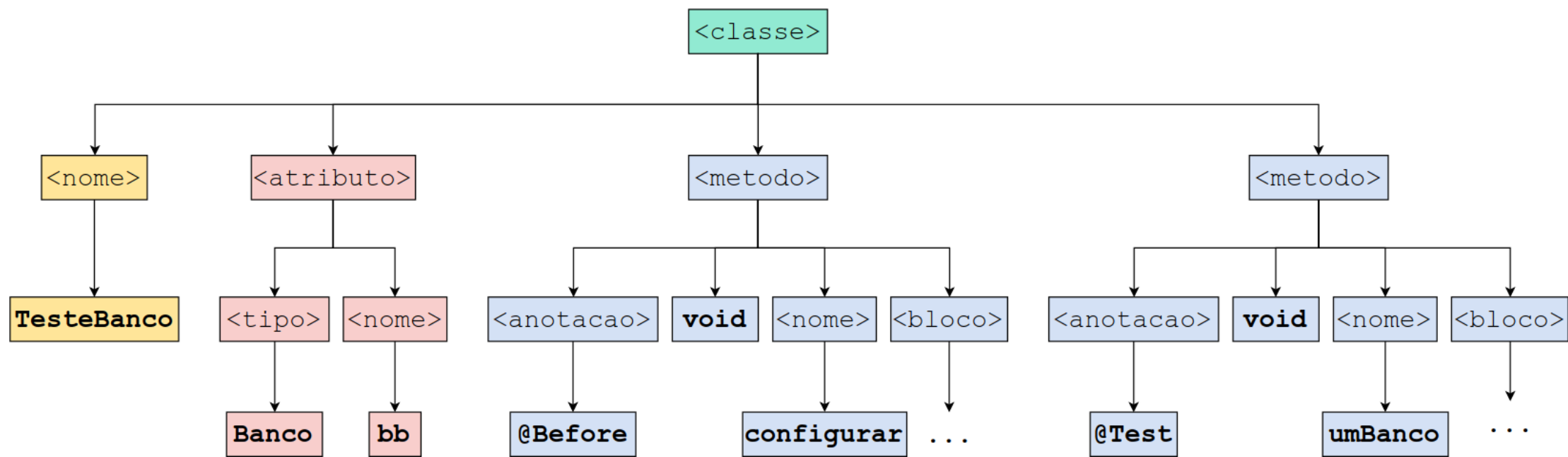
Proposta

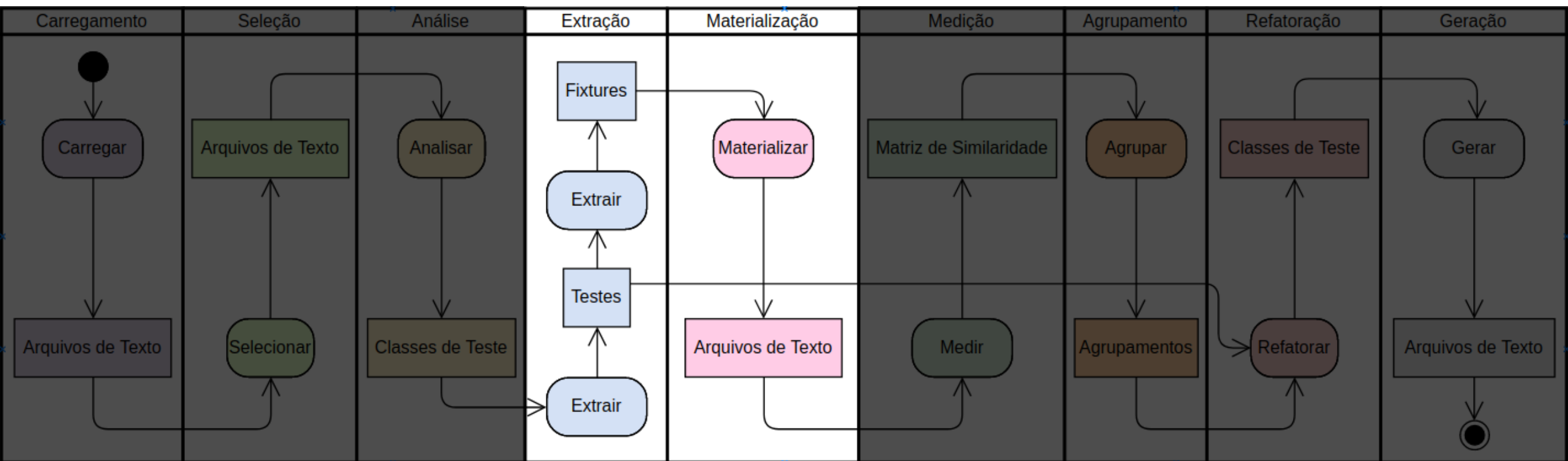
- Identificar duplicações de código de teste:
 - Que podem ser removidas através do processo de refatoração.
 - Com utilização da estratégia *implicit setup*.
- Agrupar testes com duplicações identificadas.
- Refatorar de forma automatizada o código de teste.
- Fluxo de atividades para refatoração automatizada.





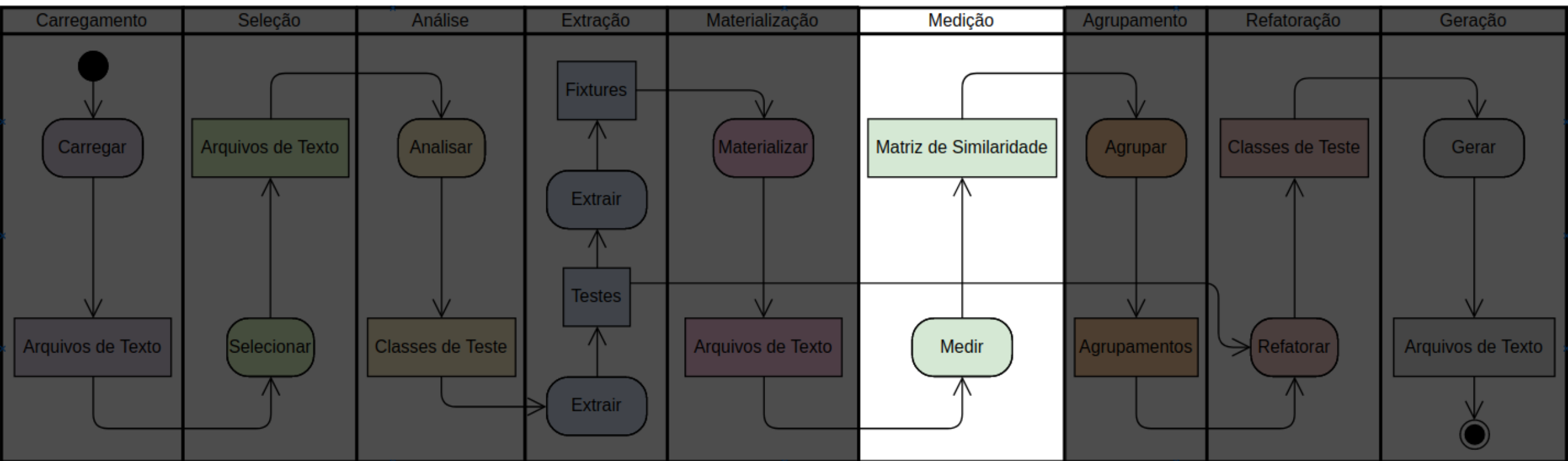






```
1 public class TesteSaqueValorDoSaldo {
2     @Test
3     public void sacarValorDoSaldo() {
4         CasaDaMoeda casaDaMoeda = new CasaDaMoeda(Moeda.BRL);
5         SistemaBancario sistema = new SistemaBancario();
6         Banco bb = sistema.criarBanco("Banco do Brasil", Moeda.BRL);
7         Agencia trindade = bb.criarAgencia("Trindade");
8         Conta joao = trindade.criarConta("João", "000.000.000-00");
9         Dinheiro cincoReais = casaDaMoeda.emitir(5);
10        joao.depositar(cincoReais);
11        Transacao saque = joao.sacar(cincoReais);
12    }
13 }
```

```
1 public class TesteSaqueMaiorQue0Saldo {
2     @Test
3     public void sacarValorDoSaldo() {
4         CasaDaMoeda casaDaMoeda = new CasaDaMoeda(Moeda.BRL);
5         SistemaBancario sistema = new SistemaBancario();
6         Banco bb = sistema.criarBanco("Banco do Brasil", Moeda.BRL);
7         Agencia trindade = bb.criarAgencia("Trindade");
8         Conta joao = trindade.criarConta("João", "000.000.000-00");
9         Dinheiro cincoReais = casaDaMoeda.emitir(5);
10        joao.depositar(cincoReais);
11        Dinheiro dezReais = casaDaMoeda.emitir(10);
12        Transacao saque = joao.sacar(dezReais);
13    }
14 }
```

	sacarValorDoSaldo	sacarMaisQueOSaldo	sacarDeChequeEspecial
sacarValorDoSaldo	1	0.9	0.7
sacarMaisQueOSaldo	0.9	1	0.8
sacarDeChequeEspecial	0.7	0.7	1

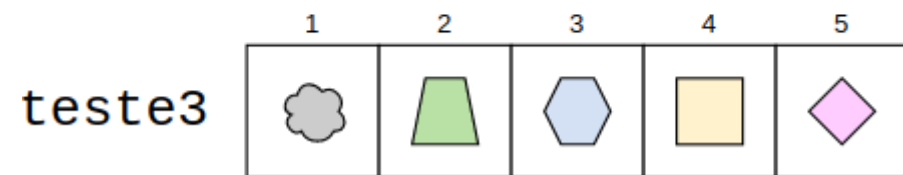
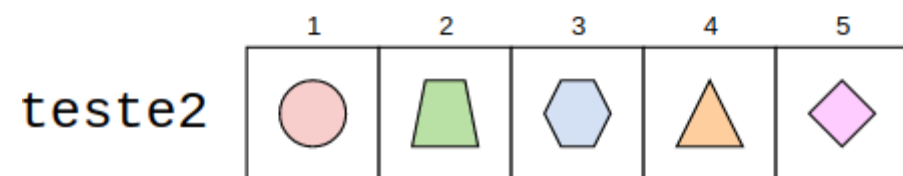
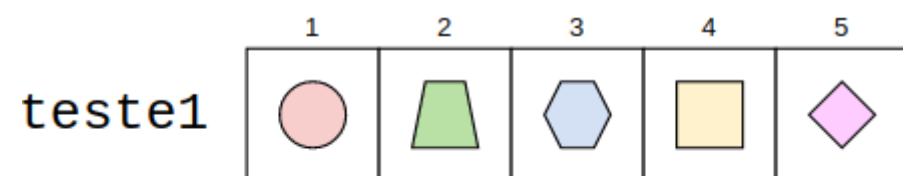
Proposta

- Grau de similaridade entre testes:
 - Deve ser alto para testes que possam ser colocados na mesma classe.
 - De modo que possam ser refatorados com a estratégia *implicit setup*.
 - Asserções devem ser desconsideradas, pois não serão refatoradas.
- Código de teste é linear:
 - Não possui estruturas de seleção.
 - Não possui estruturas de repetição.

Proposta

- *Longest Common Subsequence* (LCS).
- Normalização do código de teste.
 - Remoção de tabulações.
 - Remoção de linhas em branco.
 - Remoção de espaços desnecessários.
- Conversão do código de teste em um vetor de linhas.

```
1 public class TesteCalculadora {
2     @Test
3     public void teste1() {
4         Calculadora calculadora = new Calculadora(5);
5         calculadora.multiplicar(3);
6         calculadora.subtrair(5);
7         calculadora.somar(5);
8         calculadora.dividir(3)
9         assertEquals(5, calculadora.resultado());
10    }
11
12    @Test
13    public void teste2() {
14        Calculadora calculadora = new Calculadora(5);
15        calculadora.multiplicar(3);
16        calculadora.subtrair(5);
17        calculadora.somar(20);
18        calculadora.dividir(3)
19        assertEquals(10, calculadora.resultado());
20    }
21
22    @Test
23    public void teste3() {
24        Calculadora calculadora = new Calculadora(15);
25        calculadora.multiplicar(3);
26        calculadora.subtrair(5);
27        calculadora.somar(5);
28        calculadora.dividir(3)
29        assertEquals(15, calculadora.resultado());
30    }
31 }
```

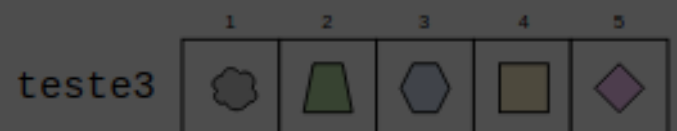




LCS



LCS



LCS

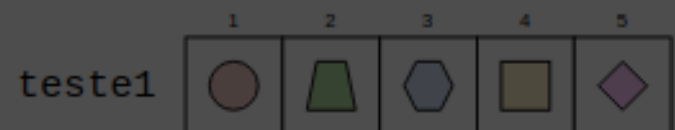


Proposta

$$\delta(\alpha, \beta) = \frac{|LCS(\alpha, \beta)| \times 2}{|\alpha| + |\beta|}$$

Proposta

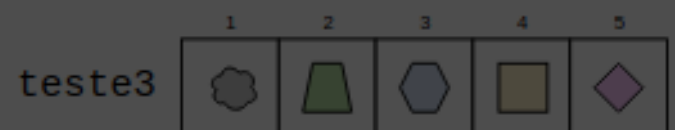
- LCS apresenta algumas deficiências:
 - Considera sequências não iniciais do código de teste.
 - Considera sequências não contíguas do código de teste.
- *Longest Common Contiguous Start Subsequence* (LCCSS).



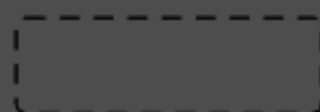
LCCSS



LCCSS

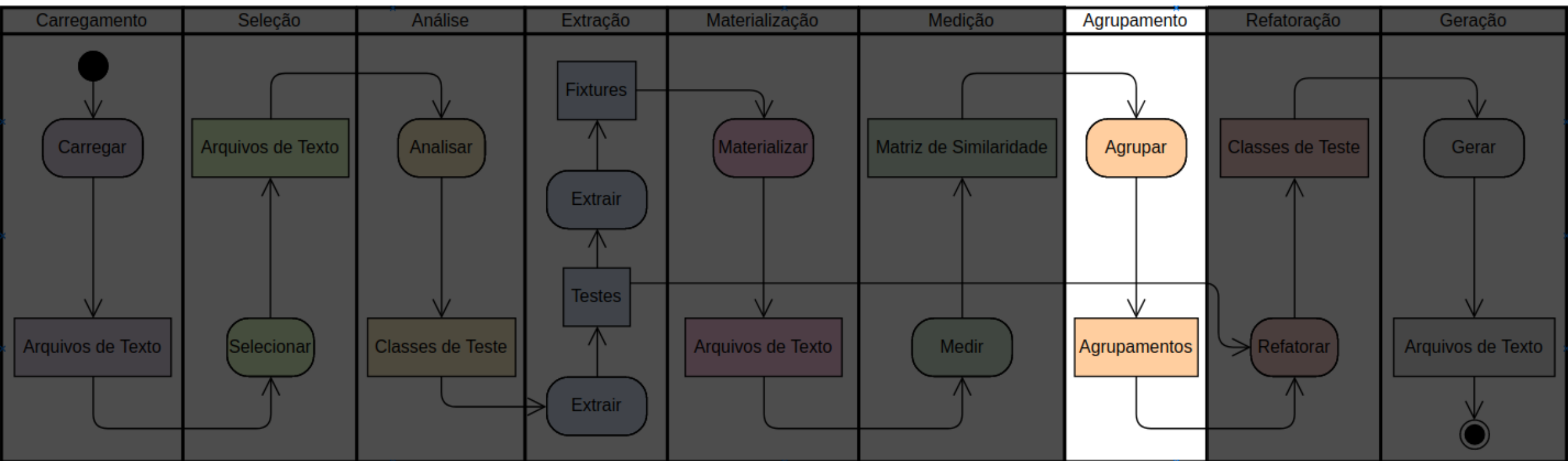


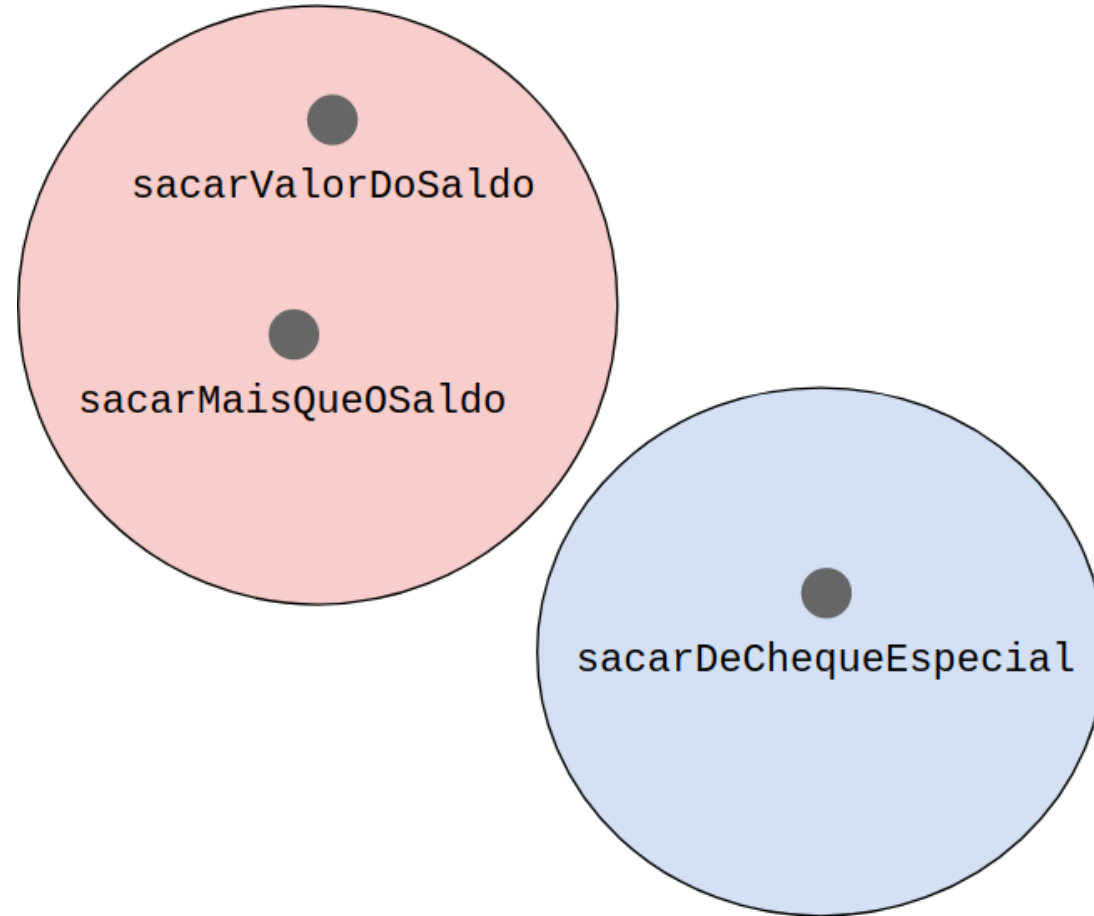
LCCSS

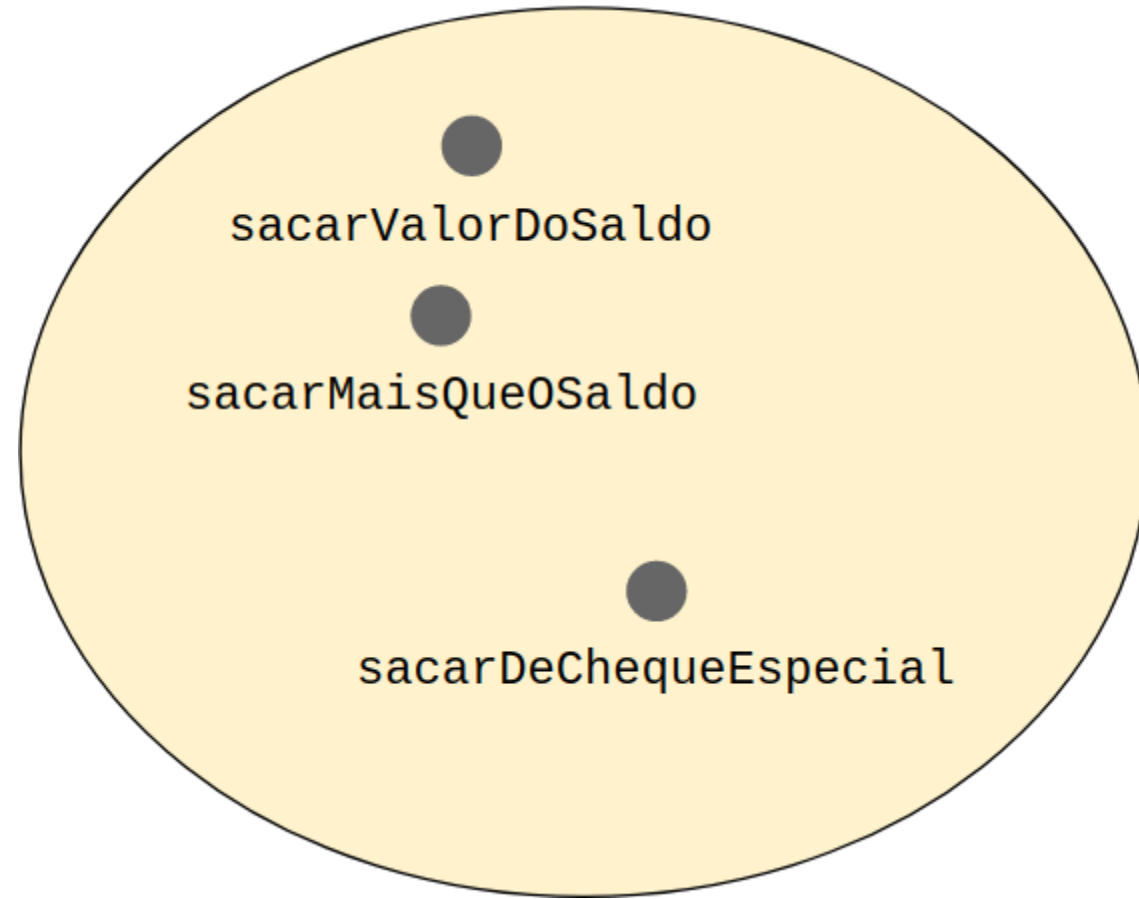


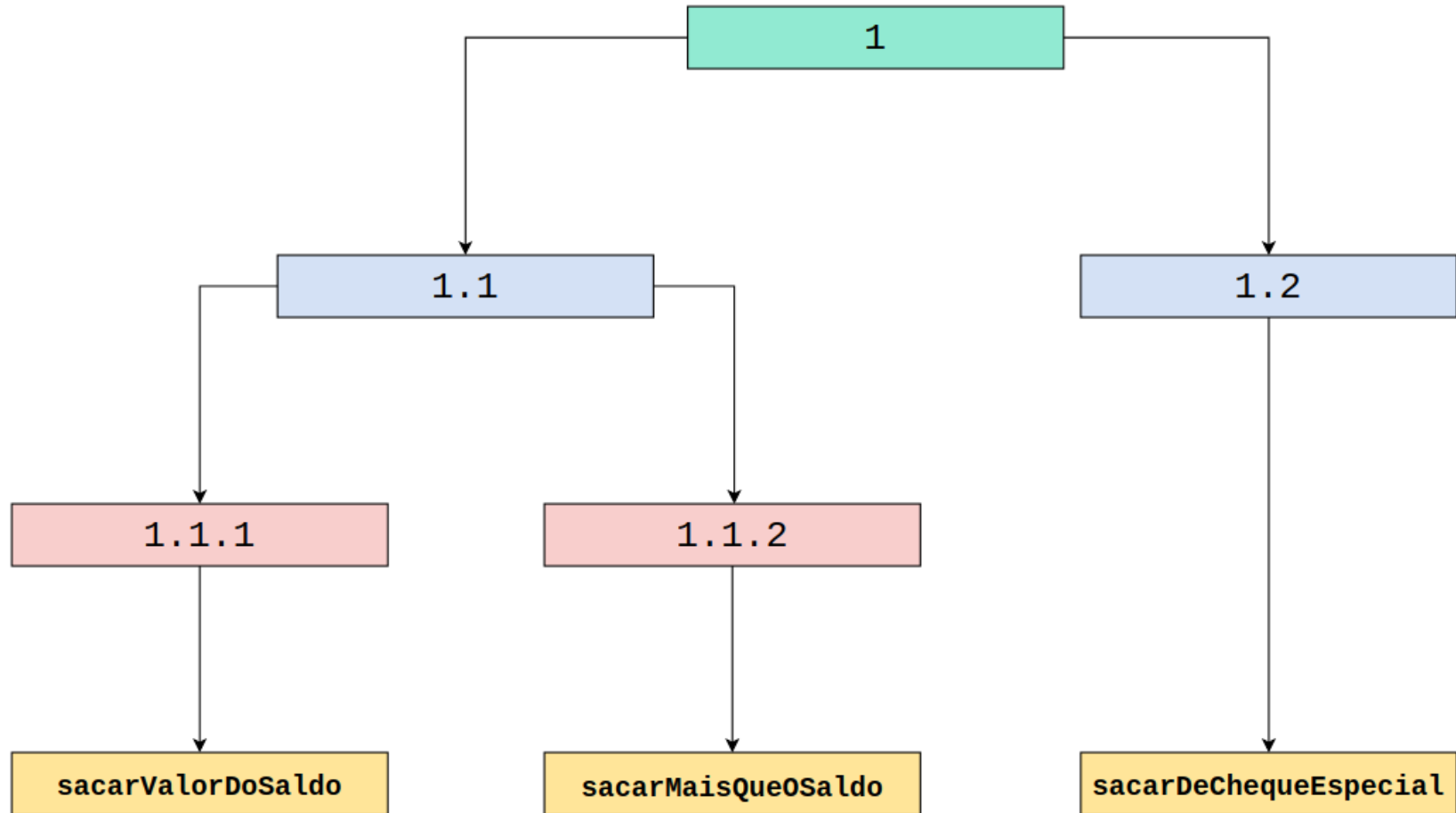
Proposta

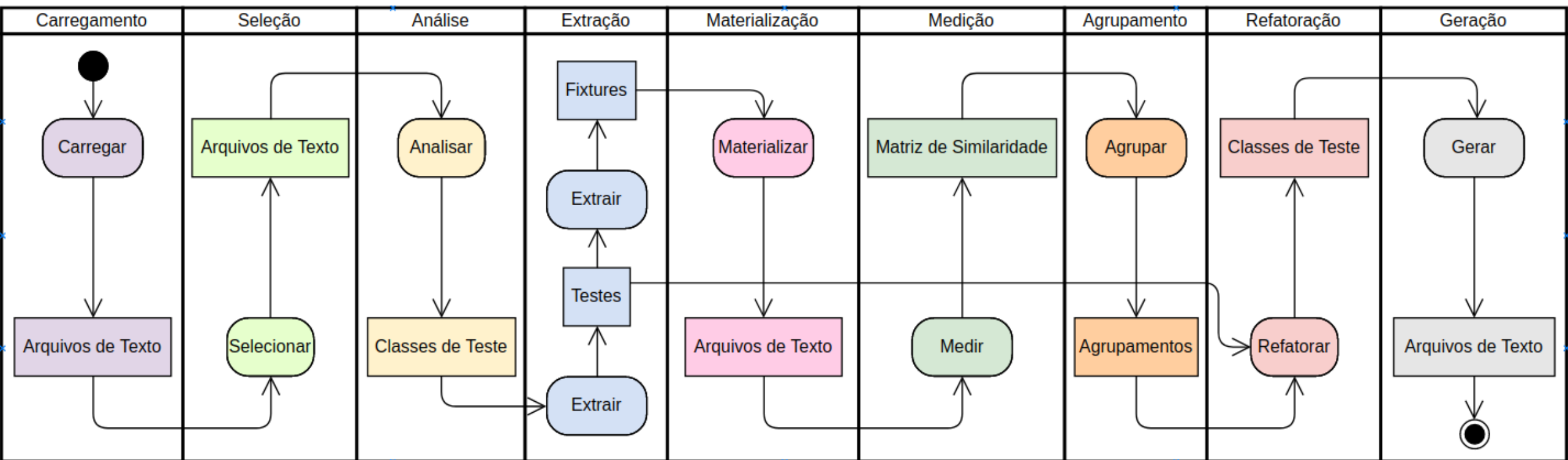
$$\delta(\alpha, \beta) = \frac{|LCCSS(\alpha, \beta)| \times 2}{|\alpha| + |\beta|}$$





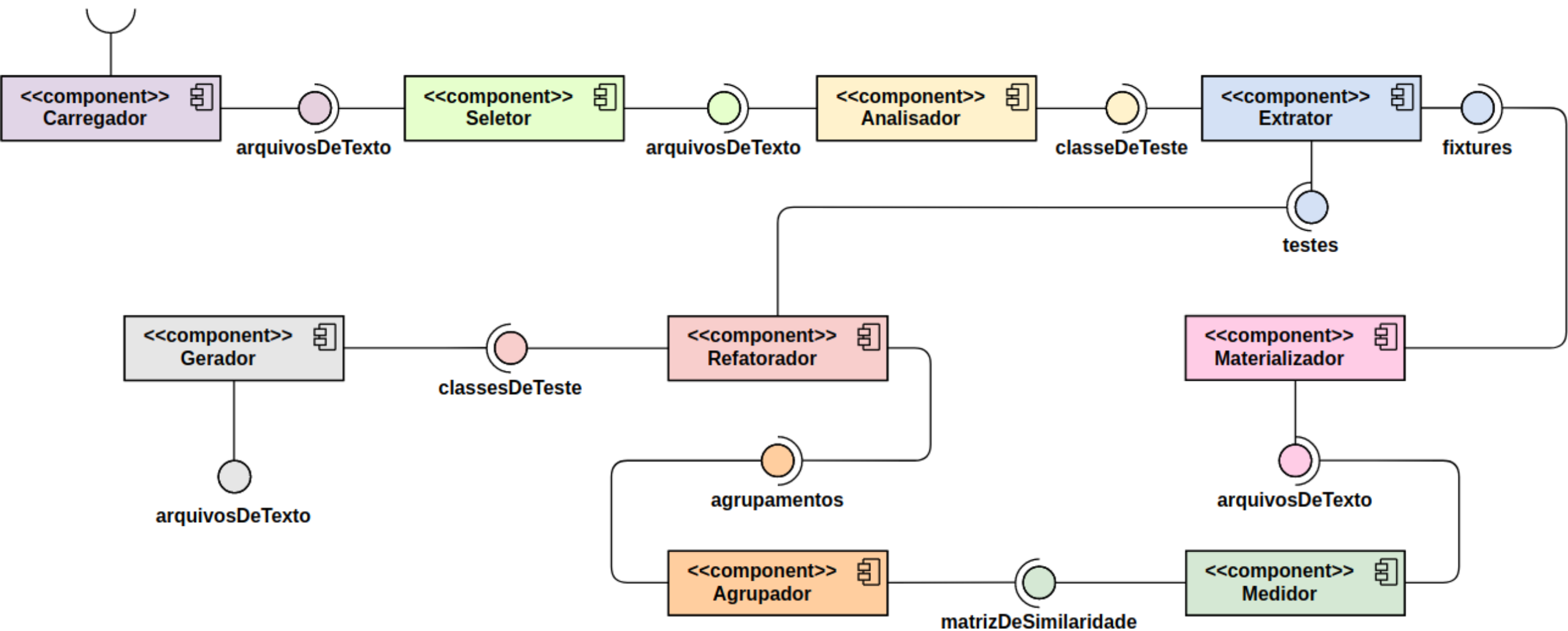






Proposta

- Desenvolvimento do *framework* Róža.
- Código aberto e disponível no GitHub.
- Desenvolvimento utilizando TDD (*Test Driven Development*).
- 87 classes e 233 testes.
- Arquitetura orientada a componentes.



Proposta

- Possui cinco métricas de similaridade:
 - LCS.
 - LCCSS.
 - JPalg.
 - Simian.
 - Deckard.
- Implementação de leitor de relatório para as três últimas.

$$\delta(\alpha, \beta) = \frac{\sum_{i=1}^n |FRAGMENTO_i^\alpha(\alpha, \beta)|}{|\alpha|}$$

TestClass4DistribuirAvaliadoresSemCoordenadorDeAreaTest.java (81.25%)	TestClass19CadastrarFichaDeAvaliacaoVerFormularioTest.java (84.78261%)	Tokens
TestClass4DistribuirAvaliadoresSemCoordenadorDeAreaTest.java(1-22)	TestClass19CadastrarFichaDeAvaliacaoVerFormularioTest.java(1-22)	31
TestClass4DistribuirAvaliadoresSemCoordenadorDeAreaTest.java(24-30)	TestClass19CadastrarFichaDeAvaliacaoVerFormularioTest.java(23-30)	8

TestClass4DistribuirAvaliadoresSemCoordenadorDeAreaTest.java

```
public class TestClass4DistribuirAvaliadoresSemCoordenadorDeAreaTest {

    @Test()
    public void distribuirAvaliadoresSemCoordenadorDeArea() {
        CorreioEletronico correioEletronico = new CorreioEletronico();
        BancoDeDados bancoDeDados = new BancoDeDados();
        Servidor servidor = new Servidor();
        Selenium selenium = new FabricaSelenium().construiCorreioEletronico.iniciar();
        bancoDeDados.criarBaseParaTestes();
        bancoDeDados.limpar();
        servidor.iniciar();
        selenium.acessar("http://localhost:8000");
        selenium.clicar("#navigationUser > li:nth-child(2)");
        selenium.digitar("gerente", "#username");
        selenium.digitar("senha", "#password");
        selenium.clicar("#login button");
        selenium.passarOMouse("#navigationUser > li > a");
        selenium.clicar("#navigationUser > li > ul > li:nth-child(1)");
        selenium.passarOMouse("#navigationPrimary > li:nth-child(1)");
        selenium.clicar("#navigationPrimary > li:nth-child(1)");
        selenium.clicar("#ui-id-2 > div > ul > li:nth-child(1)");
        String arquivoAvaliadores = "src/test/resources/casos-de-teste/arquivos/";
        selenium.clicar("#capes-abas > ul > li:nth-child(3)");
        selenium.selecionarArquivo(arquivoAvaliadores, "#capes-abas > ul > li:nth-child(3)");
        selenium.clicar("#capes-importacao-avaliadores-e-cadastrar");
        selenium.esperarAparecer(".ui-pnotify.notifySuccess");
        selenium.esperarDesaparecer(".ui-pnotify.notifySuccess");
        selenium.clicar("#capes-abas > ul > li:nth-child(5)");
        selenium.fixarValor(new Utilitarios().daquiUmMes());
        selenium.fixarValor(new Utilitarios().daquiUmMes());
        selenium.clicar("#capes-atribuicao-avaliadores-formulario");
    }
}
```

TestClass19CadastrarFichaDeAvaliacaoVerFormularioTest.java

```
public class TestClass19CadastrarFichaDeAvaliacaoVerFormularioTest {

    @Test()
    public void cadastrarFichaDeAvaliacaoVerFormulario() {
        CorreioEletronico correioEletronico = new CorreioEletronico();
        BancoDeDados bancoDeDados = new BancoDeDados();
        Servidor servidor = new Servidor();
        Selenium selenium = new FabricaSelenium().construiCorreioEletronico.iniciar();
        bancoDeDados.criarBaseParaTestes();
        bancoDeDados.limpar();
        servidor.iniciar();
        selenium.acessar("http://localhost:8000");
        selenium.clicar("#navigationUser > li:nth-child(2)");
        selenium.digitar("gerente", "#username");
        selenium.digitar("senha", "#password");
        selenium.clicar("#login button");
        selenium.passarOMouse("#navigationUser > li > a");
        selenium.clicar("#navigationUser > li > ul > li:nth-child(1)");
        selenium.passarOMouse("#navigationPrimary > li:nth-child(1)");
        selenium.clicar("#navigationPrimary > li:nth-child(1)");
        selenium.clicar("#ui-id-2 > div > ul > li:nth-child(1)");
        selenium.clicar("#capes-abas > ul > li:nth-child(3)");
        selenium.limpar("#capes-configuracao-ficha-avaliacao");
        selenium.digitar("0", "#capes-configuracao-ficha-avaliacao");
        selenium.limpar("#capes-configuracao-ficha-avaliacao");
        selenium.digitar("0", "#capes-configuracao-ficha-avaliacao");
        selenium.clicar("#capes-configuracao-ficha-avaliacao");
        selenium.esperarAparecer(".ui-pnotify.notifySuccess");
        selenium.rolarParaCima();
        selenium.passarOMouse("#navigationPrimary > li:nth-child(1)");
        selenium.clicar("#navigationPrimary > li:nth-child(1)");
        selenium.clicar("#ui-id-5");
        selenium.clicar("#reviewFormGridContainer table");
        selenium.clicar("#reviewFormGridContainer table");
    }
}
```

```

<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet href="simian.xsl" type="text/xsl"?>
<simian version="2.5.10">
  <check balanceParentheses="true" balanceSquareBrackets="true" ignoreBlocks="[0:0]" ignoreCurlyBraces="true" ignoreModifiers="true" language="Java" reportDuplicateText="true" threshold="8">
    <set lineCount="8" fingerprint="3df5c9271ee34d8b94d5f21aa05cbc2f">
      <block sourceFile="TestClass13AvaliarCoordenadorDeAreaTest.java" startLineNumber="55" endLineNumber="62"/>
      <block sourceFile="TestClass12AvaliarAvaliadorTest.java" startLineNumber="55" endLineNumber="62"/>
      <text>
<![CDATA[
  selenium.digitar("omp-capes-2019", "#password");
  selenium.clicar("#login button");
  selenium.clicar("li.pkpListPanelItem--submission > div > a");
  selenium.clicar("#privacyConsent");
  selenium.clicar("#reviewStep1 > div.section.formButtons.form_buttons > button");
  selenium.clicar("#reviewStep2 > div.section.formButtons.form_buttons > button");
  selenium.clicar("#reviewStep3 > div.section.formButtons.form_buttons > button");
  selenium.clicar("a.ok.pkpModalConfirmButton");
]]>
      </text>
    </set>
    <set lineCount="8" fingerprint="329b4c9beea2dcfa1441144dccc0491">
      <block sourceFile="TestClass30ImportarAvaliadoresCoordenadorVerAvaliadoresDeSegundaImportacaoComDadosAtualizadosTest.java" startLineNumber="30" endLineNumber="37"/>
      <block sourceFile="TestClass32ImportarAvaliadoresCoordenadorVerAvaliadoresDeSegundaImportacaoDeDuasImportacoesEmAreaDiferenteComDadosAtualizadosTest.java" startLineNumber="36" endLineNumber="43"/>
      <text>
<![CDATA[
  selenium.selecionarArquivo(arquivoAtualizado, "#capes-importacao-avaliadores-e-coordenador-selecao-arquivo");
  selenium.clicar("#capes-importacao-avaliadores-e-coordenador-form > fieldset > div.section.formButtons.form_buttons > button");
  selenium.esperarResultado("ui-pnotify.notifySuccess .ui-pnotify-text");
  selenium.passarOMouse("#navigationPrimary > li:nth-child(4) > a");
  selenium.clicar("#navigationPrimary > li:nth-child(4) > ul > li:nth-child(1) > a");
  selenium.clicar("#userGridContainer .pkp_linkaction_search");
  selenium.selecionar(17, "#userGroup");
  selenium.clicar("#userGridContainer .submitFormButton");
]]>
      </text>
    </set>
    <set lineCount="8" fingerprint="17d2368ff755af6a635f84516bed2daa">
      <block sourceFile="TestClass33ImportarAvaliadoresCoordenadorVerCoordenadorDeSegundaImportacaoDeDuasImportacoesEmAreaDiferenteComDadosAtualizadosTest.java" startLineNumber="36" endLineNumber="43"/>
      <block sourceFile="TestClass31ImportarAvaliadoresCoordenadorVerCoordenadorDeSegundaImportacaoComDadosAtualizadosTest.java" startLineNumber="30" endLineNumber="37"/>
      <text>
<![CDATA[
  selenium.selecionarArquivo(arquivoAtualizado, "#capes-importacao-avaliadores-e-coordenador-selecao-arquivo");
  selenium.clicar("#capes-importacao-avaliadores-e-coordenador-form > fieldset > div.section.formButtons.form_buttons > button");
  selenium.esperarResultado("ui-pnotify.notifySuccess .ui-pnotify-text");
  selenium.passarOMouse("#navigationPrimary > li:nth-child(4) > a");
  selenium.clicar("#navigationPrimary > li:nth-child(4) > ul > li:nth-child(1) > a");
  selenium.clicar("#userGridContainer .pkp_linkaction_search");
  selenium.selecionar(3, "#userGroup");
  selenium.clicar("#userGridContainer .submitFormButton");
]]>
      </text>
    </set>

```

000000001	dist:0.0	FILE TestClass10DistribuirAvaliadoresVerPrimeiraProducaoAtualizadaTest.java LINE:1:67 NODE_KIND:153 nVARs:31 NUM_NODE:1257 TBID:0 TEID:485
000000000	dist:0.0	FILE TestClass11DistribuirAvaliadoresVerSegundaProducaoAtualizadaTest.java LINE:1:67 NODE_KIND:153 nVARs:31 NUM_NODE:1257 TBID:0 TEID:485
000000001	dist:0.0	FILE TestClass10DistribuirAvaliadoresVerPrimeiraProducaoAtualizadaTest.java LINE:1:67 NODE_KIND:182 nVARs:31 NUM_NODE:1256 TBID:0 TEID:485
000000000	dist:0.0	FILE TestClass11DistribuirAvaliadoresVerSegundaProducaoAtualizadaTest.java LINE:1:67 NODE_KIND:182 nVARs:31 NUM_NODE:1256 TBID:0 TEID:485
000000001	dist:0.0	FILE TestClass12AvaliarAvaliadorTest.java LINE:1:64 NODE_KIND:153 nVARs:30 NUM_NODE:1198 TBID:0 TEID:452
000000000	dist:0.0	FILE TestClass13AvaliarCoordenadorDeAreaTest.java LINE:1:64 NODE_KIND:153 nVARs:30 NUM_NODE:1198 TBID:0 TEID:452
000000001	dist:0.0	FILE TestClass12AvaliarAvaliadorTest.java LINE:1:64 NODE_KIND:182 nVARs:30 NUM_NODE:1197 TBID:0 TEID:452
000000000	dist:0.0	FILE TestClass13AvaliarCoordenadorDeAreaTest.java LINE:1:64 NODE_KIND:182 nVARs:30 NUM_NODE:1197 TBID:0 TEID:452
000000001	dist:0.0	FILE TestClass10DistribuirAvaliadoresVerPrimeiraProducaoAtualizadaTest.java LINE:4:63 NODE_KIND:113 nVARs:29 NUM_NODE:1242 TBID:7 TEID:484
000000000	dist:0.0	FILE TestClass11DistribuirAvaliadoresVerSegundaProducaoAtualizadaTest.java LINE:4:63 NODE_KIND:113 nVARs:29 NUM_NODE:1242 TBID:7 TEID:484
000000001	dist:0.0	FILE TestClass12AvaliarAvaliadorTest.java LINE:4:60 NODE_KIND:113 nVARs:28 NUM_NODE:1183 TBID:7 TEID:451
000000000	dist:0.0	FILE TestClass13AvaliarCoordenadorDeAreaTest.java LINE:4:60 NODE_KIND:113 nVARs:28 NUM_NODE:1183 TBID:7 TEID:451

Roteiro

- Introdução
- Problema de Pesquisa
- Hipótese de Pesquisa
- Objetivos do Trabalho
- Fundamentação Teórica
- Estado da Arte
- Proposta
- Avaliação
- Conclusões

Avaliação

- Realização de um experimento de um sistema de avaliação de livros.
- Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES).

Avaliação

- Criação de uma matriz de controle com 46 de aceitação do sistema.
- Avaliação de um especialista sobre a possibilidade de refatoração:
 - Para cada par de teste.
 - Verificar se é possível refatorar os testes utilizando a estratégia *implicit setup*.
 - 1035 pares de teste analisados.
- Avaliação de cada métrica de similaridade:
 - Com respectivas combinações de configuração.

Avaliação

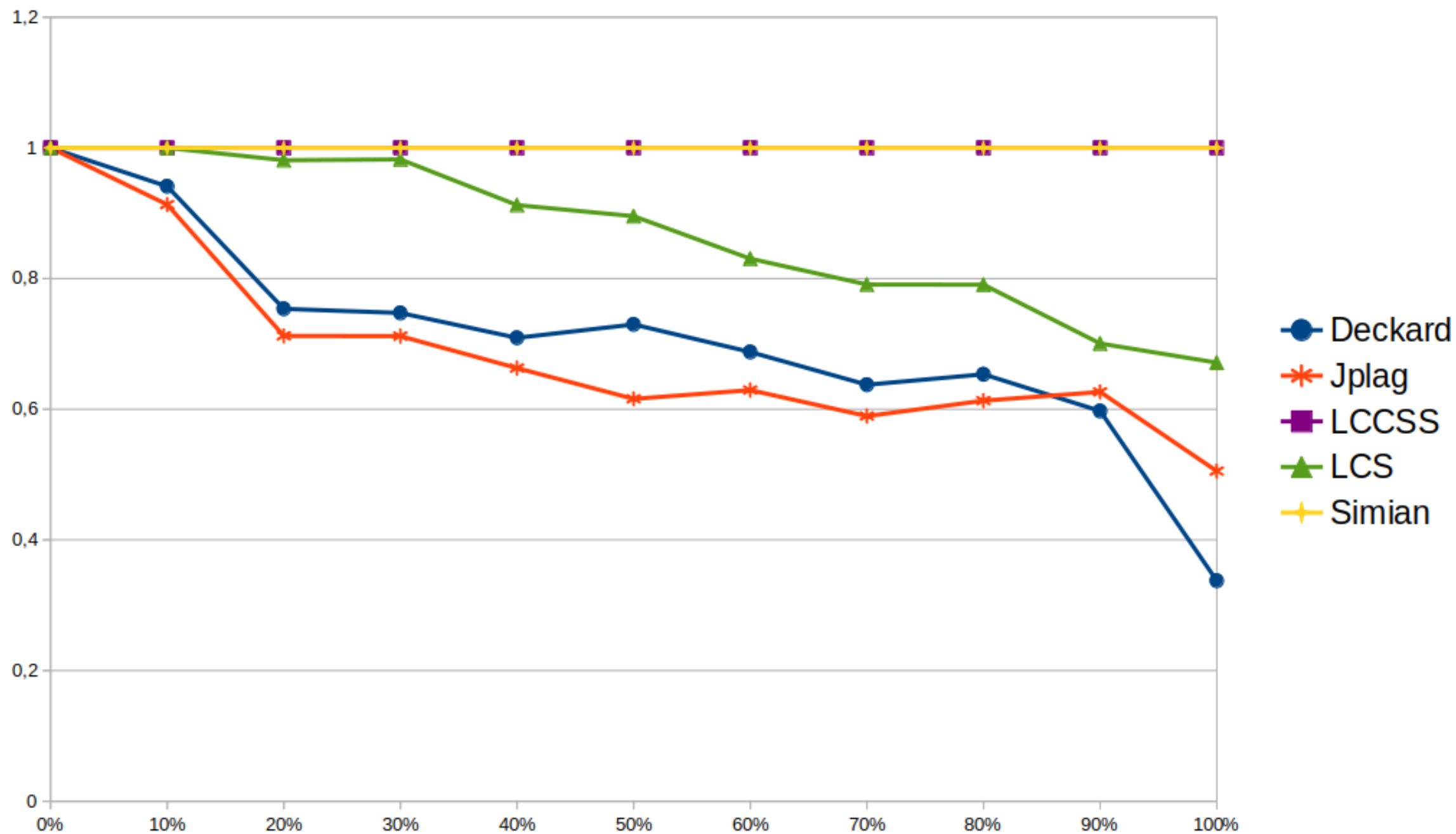
Ferramenta	Parâmetro	Descrição	Valores
JPlag	-t	Sensibilidade	1 a 50
Simian	-threshold	Limiar	2 a 15
Deckard	MIN_TOKENS	Mínimo de tokens	1 a 15
	STRIDE	Janela de sobreposição	0 a 15, inf
	SIMILARITY	Similaridade	0.9, 1.0

Avaliação

- Curva de precisão e recaptação (Baeza-Yates e Ribeiro-Neto, 1999).
- Precisão:
 - Fração dos elementos selecionados que de fato são relevantes.
- Recaptação:
 - Fração dos elementos relevantes que foram selecionados.

Avaliação

- Avaliar a classificação dos elementos relevantes de cada métrica.
- Matriz de similaridade:
 - Permite identificar os testes relevantes para cada teste.
- Métricas de similaridade:
 - Geram uma classificação dos testes mais similares para um dado teste.



Avaliação

- Ameaças da validade:
 - Utilizados apenas testes de aceitação.
 - Utilização do padrão *Arrange, Act and Assert* (AAA).
 - Não utilização do padrão *Four Phase Test*.
 - Testes sem estruturas de seleção ou repetição.

Roteiro

- Introdução
- Problema de Pesquisa
- Hipótese de Pesquisa
- Objetivos do Trabalho
- Fundamentação Teórica
- Estado da Arte
- Proposta
- Avaliação
- Conclusões

Conclusões

- Duplicação de código de teste é um problema:
 - Afeta a manutenibilidade dos testes a médio e longo prazo.
- Evitar ou remover a duplicação do código de teste:
 - Nem sempre é uma tarefa trivial se realizada manualmente.
 - Mas pode ser realizada de forma automatizada.
- Métricas de similaridade podem ser utilizadas para identificar:
 - Candidatos a refatoração.

Conclusões

- Fluxo de atividades para refatoração de classes de teste.
- *Framework* para refatoração automatizada de classes de teste.
- Nova métrica de similaridade para testes.
- Avaliação entre métrica de similaridade.

Conclusões

- Finalizar implementação do Róza.
- Incluir novas métricas de similaridade do Róza.
- Identificar métodos de agrupamento para refatoração de classes.
- Refatoração automatizada através do *implicit setup*:
 - Identificar como nomear as classes de teste geradas.
 - Eliminar métodos de teste totalmente duplicados.
 - Avaliar problemas ao inverter a ordem entre verificações e asserções.
 - Avaliar possibilidade em reordenar comandos para promover o reuso.

Conclusões

Atividade	Início	Término
Elaboração e submissão de um artigo referente à aplicação de métricas de similaridade para a identificação de duplicação de código de teste.	06/2019	08/2019
Implementação da etapa de agrupamento no framework Róža.	08/2019	09/2019
Avaliação, através da realização de experimentos, dos diferentes métodos de agrupamento implementados no Róža.	09/2019	10/2019
Elaboração e submissão de um artigo referente à utilização de métodos de agrupamento para criar grupos de testes com código em comum	10/2019	12/2019
Implementação das etapas refatoração e geração no framework Róža.	01/2020	02/2020
Avaliação, através da realização de experimentos, das diferentes combinações entre métricas de similaridade e métodos de agrupamento implementados no Róža.	02/2020	03/2020
Elaboração e submissão de um artigo referente à refatoração automatizada de classes de teste	03/2020	05/2020
Elaboração e apresentação da defesa da tese	05/2020	08/2020

Conclusões

Nome	Prazo de submissão	Qualis
Empirical Software Engineering	Fluxo contínuo	B1
Journal of Systems and Software	Fluxo contínuo	A2
Software Quality Journal	Fluxo contínuo	B1
International Conference on Software Engineering – ICSE	23/08/2019	A1
International Conference on Fundamental Approaches to Software Engineering – FASE	Ainda não definido	A2

Obrigado!